

IOb-SoC-Linux: An Open-Source, Linux-Capable RISC-V SoC Template for IP Core Development

Artur Nóbrega ¹, João Barreiros Rodrigues ^{2*}, Jaime Aguiar ^{1,2}, Jiao Li ³ and José T. de Sousa ^{2*}

¹ IObundle, Lda., 1000-029 Lisbon, Portugal; artur.nobrega@tecnico.ulisboa.pt (A.N.); jaime.a.aguiar@tecnico.ulisboa.pt (J.A.)

² INESC-ID, INESC INOV, Instituto Superior Técnico, Universidade de Lisboa, 1000-029 Lisbon, Portugal

³ Microelectronics Research Development Center, School of Mechatronic Engineering and Automation, Shanghai University, Shanghai, 200444, China; lijiaoshu@shu.edu.cn

* Correspondence: joao.b.rodrigues@inov.pt (J.B.R.); jose.t.de.sousa@tecnico.ulisboa.pt (J.T.d.S.)

Abstract

IOb-SoC-Linux is an open-source framework for configuring and generating Linux-capable System-on-Chip (SoC) designs targeting RISC-V processors. Built on the Py2HWSW hardware/software co-design framework, it provides a specialized SoC template that contrasts with bare-metal-focused alternatives, offering a reproducible environment for the design, integration, and testing of Intellectual Property (IP) cores under a full Linux software stack. The primary contribution of IOb-SoC-Linux is an IP Core Linux Driver Development tool that automates the generation of Linux device drivers, device tree descriptions, and associated documentation, significantly accelerating the production of high-quality, reusable IP cores. The platform supports the open-source the VexRiscv and VexiiRiscv RISC-V processors. Beyond the processor subsystem, it integrates the complete infrastructure required for Linux execution: memory management unit, memory controller, peripheral buses, interrupt controllers, timer, UART interface, storage controllers, and configurable IP peripherals. To improve interoperability and reuse, IOb-SoC-Linux adopts the IP-XACT standard (IEEE 1685) alongside the FuseSoC package manager, enabling standardized metadata descriptions and structured package management for hardware IP blocks. This allows hardware components to be readily discovered, configured, and integrated into both the IOb-SoC-Linux environment and the broader open-source hardware ecosystem. The system is designed for portability across multiple FPGA platforms and to be synthesized as an ASIC, demonstrating its suitability for both prototyping and silicon-ready SoC development.

Keywords: design automation; hardware/software co-design; embedded Linux; RISC-V; Linux register-access driver generation; IP reuse; IP-XACT; FPGA

1. Introduction

Modern embedded systems increasingly run a full operating system—from network routers to industrial controllers and edge gateways—while still demanding rapid integration of custom Intellectual Property (IP) cores. Reduced Instruction Set Computer V (RISC-V) has become a key enabler of such systems, offering a modular, extensible, and royalty-free open-standard foundation for both academic and industrial System-on-Chip (SoC) design. Assembling a Linux-capable SoC around it, however, remains a fragmented process: reusable IP, standardized software interfaces, and maintainable hardware abstractions must be combined by hand and kept consistent as the design evolves.

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Electronics* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

Three recurring obstacles make this integration costly. First, the *software-synchronization problem*: every custom peripheral is described three times over—in the Register Transfer Level (RTL), in the Device Tree, and in a Linux kernel driver—and these descriptions drift apart whenever a register map or interrupt line changes. Second, the *platform-assembly problem*: running Linux rather than bare-metal firmware requires a specific set of hardware and low-level software components—an Memory Management Unit (MMU), standards-compliant interrupt and timer controllers, an Supervisor Binary Interface (SBI) implementation, an early console, and a root filesystem—that bare-metal SoC templates do not provide. Third, the *interoperability problem*: hardware blocks integrated in one framework rarely carry the standardized metadata needed to reuse them in another.

We present IOB-SoC-Linux, an open-source SoC generation framework that addresses these three obstacles for Linux-capable RISC-V systems, extending the earlier design of [1]. Built on top of the Py2HWSW hardware/software co-design infrastructure, it replaces the bare-metal-focused SoC template with one organized around full operating system support, and automates both the hardware integration and the generation of the accompanying software stack.

The framework's central goal is to make adding a register-access IP core follow from its hardware description instead of being a separate manual step: the register interface, address space, and interrupt line declared for a peripheral are the primary description from which its register-access Linux kernel driver, Device Tree entry, and configuration files are generated, supplemented only by a small amount of integration metadata such as a Device Tree `compatible` string and interrupt properties, plus any peripheral-specific semantics added by hand on top of the generated module. Around this pipeline, IOB-SoC-Linux supplies a standardized SoC infrastructure—interrupt and timer controllers, memory subsystem, and peripheral buses—sized for Linux operation, supports multiple open-source RISC-V cores to span a range of performance and resource points, and remains portable across Field-Programmable Gate Array (FPGA) platforms and Application-Specific Integrated Circuit (ASIC) synthesis flows. For interoperability beyond its own environment, its peripheral IP cores are additionally exported as IP-XACT (IEEE 1685) descriptions and FuseSoC packages, keeping hardware blocks discoverable and reusable across other toolchains.

The main contributions of this work are:

- **Automatic generation of Linux register-access drivers and integration artifacts.** From a peripheral's Py2HWSW Control and Status Register (CSR) description—register addresses, widths, and access modes—together with a small amount of integration metadata, the `iob_linux_device_drivers` module emits a loadable register-access Linux kernel driver, its user-space library, Device Tree snippet, C test suite, and documentation alongside the generated RTL, removing the manual synchronization a new IP core otherwise requires. Peripheral-specific semantics—register side effects, first-in-first-out ports, self-clearing bits, multi-register sequences, and interrupt-acknowledgement behaviour—remain a manual addition on top of the generated module.
- **Standards-based packaging.** A generic exporter emits, for a packaged core, an IEEE 1685 IP-XACT description and a versioned FuseSoC `.core` package, making it discoverable and reusable across open-source and commercial Electronic Design Automation (EDA) toolchains alike. The three peripheral IP cores packaged this way in the present work—IOB-Cache, IOB-UART16550, and IOB-Eth—are released to the open-source hardware community as MIT-licensed blocks; IOB-UART16550 and IOB-Eth are maintained, mainline-driver-compatible reimplementations of the widely used OpenCores `uart16550` [2] and `ethmac` [3] designs, and IOB-Cache is described in a separate peer-reviewed study [4].

- **A Linux-ready SoC template.** We combine a configurable VexRiscv/VexiiRiscv subsystem with external `iob_clint` and `iob_plic` controllers, a 16550-class console, and the memory and bus infrastructure required by upstream OpenSBI and Linux, portable across FPGA boards and an ASIC synthesis flow.
- **Hierarchical verification.** We exercise the flow from per-core linting and coverage analysis, through bare-metal and Linux-mode system simulation, to on-board FPGA execution of the generated driver test suites.

2. Related Work

We position IOB-SoC-Linux against two groups of related efforts: general-purpose SoC generator frameworks for which Linux is one software target among several, and platforms built specifically around a Linux-capable RISC-V core. For each system we summarize its design and then close with a paragraph contrasting it with the concrete workflow IOB-SoC-Linux implements. That workflow has two halves. The `iob_linux_device_drivers` and `iob_system_dts.py` tooling generates Linux kernel drivers, Device Tree sources, C test suites, and documentation directly from hardware CSR descriptions; the `fusesoc.py` and `ipxact_gen.py` exporters produce, respectively, Vendor:Library:Name:Version (VLNV)-tagged FuseSoC `.core` packages and IEEE 1685-compliant IP-XACT descriptions for its peripheral IP, currently IOB-Cache, IOB-UART16550, and IOB-Eth. The comparisons that follow are based on each project's public documentation and documented workflow as surveyed in 2025; individual extensions or third-party tooling may narrow some of the gaps noted.

2.1. LiteX

LiteX [5] is an open-source SoC builder and IP library in which hardware components are described using the Migen Python internal domain-specific language. It supports a wide range of open-source and commercial soft-core processors and a Wishbone/AXI-based interconnect, and it generates C header files exposing peripheral CSR maps to bare-metal and Linux software; for its own catalog of standard peripherals it additionally exports a Linux Device Tree Source (DTS) file from the SoC's JSON description via the `litex_json2dts_linux.py` tool.

However, this generation pipeline is scoped to peripherals already known to LiteX: Linux kernel drivers for new, custom IP cores must still be written by hand and integrated into the Device Tree template manually, and no IP-XACT or equivalent standardized metadata is produced. IOB-SoC-Linux automates this remaining gap end to end: for any peripheral described in Py2HWSW, `iob_linux_device_drivers` generates the kernel module itself—exposing `sysfs`, `/dev`, and `ioctl` interfaces—together with its Device Tree snippet, C test suite, and documentation, and additionally emits a FuseSoC `.core` package and an IP-XACT description that LiteX's flow has no equivalent for.

2.2. Chipyard

Chipyard [6] is a Chisel/RocketChip-based, generator-driven framework from UC Berkeley for the design, FPGA-accelerated simulation, and ASIC implementation of custom RISC-V SoCs, supporting in-order Rocket and out-of-order BOOM cores alongside custom accelerators connected through the Diplomacy parameter-negotiation framework. Diplomacy's two-phase elaboration automatically emits a system-wide Device Tree from the elaborated hardware graph, but integrating a new accelerator still requires manually writing its Linux kernel driver and supplying the generator with the appropriate DTS compatible-string binding; the framework does not automate driver generation from the hardware description, nor does it target IP-XACT interoperability.

IOb-SoC-Linux covers the step Diplomacy leaves manual: its `iob_linux_device_drivers` module generates the kernel driver itself from the same CSR description used to build the Device Tree snippet (via `iob_system_dts.py`), rather than only the DTS binding, and packages the resulting IP core as a FuseSoC `.core` together with an IP-XACT description for reuse outside the Chipyard/Diplomacy ecosystem.

2.3. X-HEEP

X-HEEP [7] is an open-source, configurable and extendible single-core RISC-V microcontroller from EPFL, built on IP from the PULP platform, OpenHW Group, and OpenTitan projects, and extendible through the eXtensible Accelerator InterFace. X-HEEP targets bare-metal and Real-Time Operating System (RTOS) use on ultra-low-power edge devices rather than a full Linux software stack, and does not automate kernel driver, Device Tree, or IP-XACT metadata generation for newly attached peripherals.

IOb-SoC-Linux targets the opposite end of this design space: full Linux execution is a primary goal, which is why its default VexRiscv/VexiiRiscv processor subsystem moves the Core-Local Interruptor (CLINT) and Platform-Level Interrupt Controller (PLIC) out of the CPU wrapper into external `iob_clint` and `iob_plic` peripherals, so that switching cores needs no wrapper changes to route those signals out and Linux still sees the upstream OpenSBI-compatible bindings, and why every attached peripheral automatically receives a generated Linux kernel driver, Device Tree entry, and IP-XACT/FuseSoC package rather than the bare-metal register headers X-HEEP-class microcontrollers rely on.

2.4. Cheshire

Cheshire [8] is a lightweight, modular 64-bit Linux-capable RISC-V host platform from ETH Zürich, built around the CVA6 core and designed as a minimal base for plugging in domain-specific accelerators. Written in SystemVerilog, it features a fully digital, low-pin-count HyperRAM Dynamic Random-Access Memory (DRAM) interface, a last-level cache configurable as scratchpad memory, and a DMA engine for efficient accelerator-to-memory data movement; it is also the host platform on which Basilisk (below) is built.

As with the frameworks above, integrating a new peripheral into Cheshire requires hand-written Linux drivers and Device Tree entries, with no automated code-generation or IP-XACT export pipeline. IOb-SoC-Linux instead treats this integration step as a generated artifact of the hardware description: adding a peripheral to the Py2HWSW-based `iob_system_linux` core automatically produces its driver, DTS entry, test suite, and IP-XACT/FuseSoC packaging, without requiring the accelerator-specific, hand-assembled integration Cheshire is designed around.

2.5. Basilisk SoC

Basilisk [9] is an end-to-end open-source, Linux-capable RISC-V SoC taped out in IHP's open 130 nm CMOS technology. Built on the Cheshire SoC platform, it integrates an RV64GC CVA6 processor core, a fully digital HyperRAM DRAM controller, and a rich peripheral set including USB 1.1 and VGA. A key contribution of Basilisk is the improvement of the open-source EDA flow: by developing the SVase SystemVerilog pre-elaborator and optimizing Yosys synthesis, the authors achieved a $2.3\times$ improvement in peak clock speed and a $1.6\times$ reduction in cell area. A subsequent silicon measurement report [10] confirms a 34 mm^2 , 2.7 MGE fabricated chip reaching 62 MHz at nominal voltage with a peak energy efficiency of 18.9 DP MFLOP/s/W.

While Basilisk demonstrates end-to-end open-source silicon with a competitive EDA flow, it is hand-assembled and does not provide a systematic co-design framework for integrating and generating new IP cores together with their associated Linux software stack. IOb-SoC-Linux addresses this gap by automating the generation of device drivers, Device

Tree descriptions, C test suites, and documentation directly from hardware CSR descriptions via `iob_linux_device_drivers`, enabling faster and more consistent IP integration under Linux than Basilisk's hand-assembled flow.

2.6. HULK-V SoC

HULK-V [11] is an open-source heterogeneous, ultra-low-power, Linux-capable RISC-V SoC targeting Internet of Things (IoT) applications that demand both full Operating System (OS) support and high energy efficiency. It couples a 64-bit RISC-V host processor with an 8-core Programmable Multi-Core Accelerator (PMCA), delivering up to 13.8 GOps and 157 GOps/W, with DSP and ML task acceleration of up to 112× over the host. Implemented in GlobalFoundries 22 nm FDX technology, the design operates within a 250 mW power envelope and exposes up to 512 MB of DRAM via a fully digital HyperRAM memory hierarchy. The Linux software stack supports OpenMP host-to-PMCA offload, enabling productive parallel programming on a power-constrained platform.

HULK-V focuses on a fixed heterogeneous architecture optimized for energy efficiency, with the PMCA tightly coupled to the host processor, and, like the platforms above, does not automate driver or metadata generation for its peripherals. In contrast, IOB-SoC-Linux targets a configurable SoC template supporting multiple RISC-V cores (VexRiscv, VexiiRiscv), where any peripheral integrated into `iob_system_linux`—not a fixed accelerator topology—automatically receives a generated Linux driver, Device Tree entry, and IP-XACT/FuseSoC package.

2.7. OpenTitan and Metadata Standardization

OpenTitan [12] is an open-source silicon root-of-trust project stewarded by lowRISC in partnership with Google, ETH Zürich, Nuvoton, and G+D Mobile Security, built around the Ibex RISC-V core together with cryptographic coprocessors, a hardware key hierarchy, and secure-boot infrastructure. Unlike the platforms discussed above, OpenTitan targets a dedicated, security-hardened firmware environment rather than general-purpose Linux execution, and uses a Bazel- and hjson-based register description toolchain (`reggen`) that auto-generates C headers and Driver Interface Function stubs from hardware register definitions. This toolchain demonstrates the value of generating software artifacts directly from a structured hardware description, but it is project-specific to OpenTitan, targets a custom firmware stack rather than the Linux kernel, and does not interoperate with the IP-XACT (IEEE 1685) standard.

Commercial EDA vendors address IP-metadata standardization through proprietary tools such as Arm Socrates [13], which allows designers to browse, configure, and build Arm and third-party IP, exporting IP-XACT descriptions alongside generated RTL. Such tools demonstrate the industrial value of IP-XACT-based metadata but are tied to commercial licensing and vendor-specific IP catalogs. In the open-source domain, FuseSoC [14] provides a lightweight package manager and tool-agnostic build abstraction for Hardware Description Language (HDL) cores, resolving dependencies across more than a dozen simulation, synthesis, and lint tools, but it does not itself define or export a standardized IP metadata format such as IP-XACT.

IOB-SoC-Linux bridges these two worlds: its `fusesoc.py` export generates, for a packaged core, a FuseSoC `.core` package with an `iobundle.py2hws: name: version` VLNV identifier, filesets, and tool-specific targets, while a companion `ipxact_gen.py` exporter produces IEEE 1685-compliant IP-XACT metadata—carrying the core's parameters, ports, bus interfaces, and register map—intended to interoperate with commercial tools such as Vivado IP Integrator and Intel Platform Designer. The exporters are generic to any Py2HWSW core; the peripheral IP packaged and checked this way in the present work

is IOb-Cache, IOb-UART16550, and IOb-Eth, and broadening this coverage is ongoing. Unlike OpenTitan's project-specific reggen flow, this metadata is coupled directly to the automated Linux kernel driver and Device Tree generation pipeline described in Section 4.3, and applies to register-access peripheral IP rather than to a single fixed SoC.

3. Background

This section reviews the two foundations of IOb-SoC-Linux: the platform requirements that a RISC-V SoC must meet to run a general-purpose Linux kernel, and the Py2HWSW co-design framework that IOb-SoC-Linux extends to satisfy them automatically.

3.1. Linux on RISC-V: Platform Requirements

Running a general-purpose Linux kernel on a RISC-V SoC—rather than bare-metal or RTOS firmware—imposes a well-defined set of hardware and low-level software requirements. These requirements shape every structural decision in IOb-SoC-Linux and are summarized below, each paired with the corresponding design response.

- **Privilege modes and trap delegation.** The kernel runs in supervisor (S) mode, unprivileged application code runs in user (U) mode, and machine (M) mode is expected to be occupied by firmware that delegates most traps and interrupts to S-mode [15]. IOb-SoC-Linux uses cores (VexRiscv, VexiiRiscv) that implement the full M/S/U privilege hierarchy.
- **Virtual memory.** Linux requires a hardware MMU; on 32-bit RISC-V this is the Sv32 paging scheme [15]. Both supported cores are configured with Sv32, and the generated DTS advertises `mmu-type = "riscv,sv32"`.
- **Base Instruction Set Architecture (ISA).** The mainline RISC-V Linux port requires at least the integer, multiply/divide, and atomic extensions, with the compressed extension strongly recommended for code density. IOb-SoC-Linux targets RV32IMAC.
- **Standard timer and software interrupts.** The kernel obtains its timebase and cross-hart signalling through the SBI timer and inter-processor-interrupt calls [16], which M-mode firmware services using the CLINT. IOb-SoC-Linux provides an external `iob_clint` exposing the upstream `riscv,clint0` binding.
- **External interrupt routing.** Device interrupts must be aggregated, prioritized, and delivered to per-hart machine- and supervisor-mode contexts by a PLIC [15]. IOb-SoC-Linux provides an external `iob_plic` exposing the `riscv,plic0` binding.
- **M-mode firmware boot contract.** A supervisor-mode payload must be launched by an M-mode SBI implementation; IOb-SoC-Linux uses OpenSBI [17], which initializes the platform, installs the trap and SBI call handlers, and hands control to the kernel.
- **Console and root filesystem.** Bring-up and user space require an early console and a mountable root filesystem. IOb-SoC-Linux pairs a 16550-class Universal Asynchronous Receiver-Transmitter (UART), served by the mainline 8250 driver, with a Buildroot/BusyBox `initramfs`.
- **Discoverable hardware description.** The kernel binds drivers from a Device Tree rather than by probing fixed addresses, making the Device Tree the hardware/software contract that the toolchain of Section 4.3 generates automatically.

Existing bare-metal SoC templates satisfy few of these requirements out of the box; supplying them consistently—and keeping the software artifacts synchronized with the hardware as it evolves—is the problem IOb-SoC-Linux addresses.

3.2. Py2HWSW

Py2HWSW is an open-source Python-based hardware/software co-design framework that automates the generation of portable, lint-friendly Verilog for both FPGA and ASIC

targets; the complete project is available in [18]. Hardware modules are described using Python or JSON dictionaries and internally represented as structured `iob_core` objects, augmenting existing RTL workflows by generating readable and reusable HDL that integrates naturally with conventional hardware projects.

The framework follows a two-stage workflow. During the setup phase, high-level core descriptions are processed into Verilog source files and auxiliary build artifacts. In the subsequent build phase, these generated sources are synthesized into FPGA bitstreams or ASIC netlists. The Verilog backend is modular, relying on specialized generators for parameters, interfaces, combinational logic, and interconnect structures assembled into complete modules. A key architectural feature is the use of standardized interfaces: hardware cores expose configurable Python parameters and return structured Core Dictionaries, enabling concise and scalable hardware descriptions while preserving flexibility for advanced customization. Each core is self-contained within its own setup directory, allowing reusable IP blocks and peripherals to be independently developed, configured, and integrated.

To ensure reproducibility across development environments, Py2HWSW uses the Nix package manager to provision a deterministic toolchain including Verilator, Icarus Verilog, Yosys, RISC-V toolchains, and FuseSoC, and additionally provides a reusable library of hardware IP cores and peripherals.

4. IOB-SoC-Linux Extensions to the Toolchain

The IOB-SoC-Linux toolchain is delivered as two coupled pieces of Python that build directly on the Py2HWSW framework of Section 3.2: a framework-level extension, `iob_system_linux`, that turns the generic Py2HWSW SoC template into a Linux-capable one, and a repository-level template, `soc_linux.py`, that a user edits to describe a concrete SoC instance. Both use the same parent/child core mechanism, so the generation, build, and packaging machinery already provided by Py2HWSW is inherited unchanged; only the Linux-specific behavior is added at each level. This section details these two scripts, the driver and metadata generation pipeline they drive, and the build flow that wraps them; the SoC architecture they produce is described in Section 5.

Figure 1 gives the overall picture. In the user's sandbox, the `soc_linux.py` SoC template is combined with user-supplied configuration and IP libraries to produce a concrete `soc_linux.py` SoC specification. Py2HWSW and its Linux integrator (the `iob_system_linux` core) then elaborate this specification—iteratively, as hardware and drivers are refined—into the IOB-SoC-Linux system together with a matching tester that exercises it through stimulus/response pairs.

4.1. Py2HWSW Integration: the `iob_system_linux` Core

Earlier iterations of IOB-SoC-Linux [1], and its OpenCryptoLinux successor, carried the Linux specialization as custom generator code inside the SoC repository itself, tightly bound to a particular CPU and to a hand-maintained build system. That specialization has now been generalized and upstreamed into the Py2HWSW core library as `iob_system_linux`, a standard Py2HWSW core that extends the framework's baseline `iob_system` template through native core inheritance rather than through repository-local patches.

`iob_system_linux` is a Py2HWSW `setup()` function that declares `iob_system` as its parent and then overrides a small set of parameters and attributes:

- **System configuration.** It fixes the defaults required for a Linux target—external DRAM enabled, internal memory disabled, Ethernet enabled, a 64 MiB memory map, a 64 KiB boot Read-Only Memory (ROM), and firmware linked at the base of DRAM—and selects a Linux-capable RISC-V core (VexRiscv by default, VexiiRiscv selectable) with MMU support.

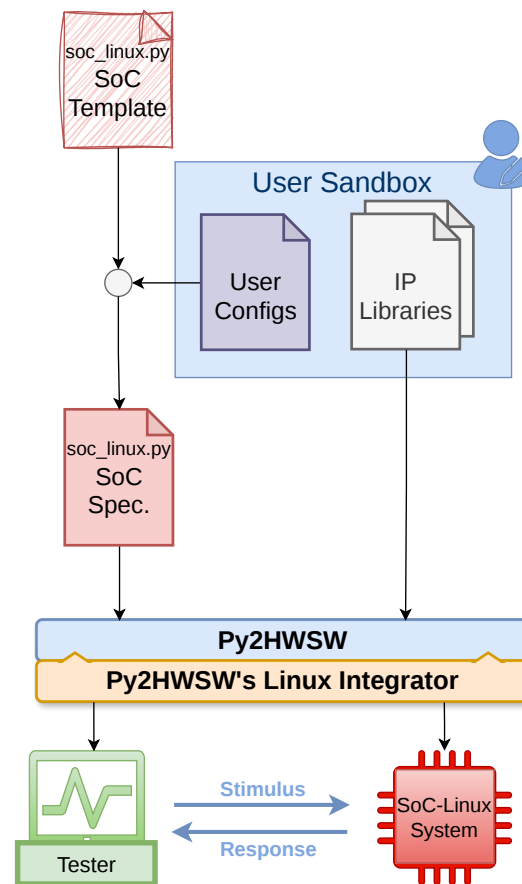


Figure 1. IOb-SoC-Linux toolchain overview

- Interrupt architecture.** The baseline `iob_system` already exposes the CLINT and PLIC as external `iob_clint` and `iob_plic` peripherals rather than CPU-internal units. `iob_system_linux` keeps this arrangement and supplies its own interrupt-vector wiring, placing the timer, the Ethernet Media Access Controller (MAC), and the three UARTs at fixed PLIC source identifiers 1–5.
- Console peripherals.** It replaces the baseline `iob_uart` with three `iob_uart16550` instances: one console UART bound to the board's serial port, used for the Linux console and for loading firmware images at boot, plus a second pair of instances wired to each other in an internal loopback and used only to self-test the UART driver and hardware, independently of the system console. All three instances are compatible with the mainline Linux 8250 driver.
- Device Tree generation.** It invokes `generate_dts` from the bundled `iob_system_dts.py` script to emit the system-level DTS—CPU node, MMU type, memory node, kernel bootargs, and the soc bus—which then `#includes` a `peripherals.dtsi` file carrying the per-peripheral snippets produced by the pipeline of Section 4.3.
- Software integration.** It attaches the `iob_linux` software module—the kernel, firmware, and root-filesystem build support—to the system. Per-peripheral driver generation is not driven from here but from each peripheral's own description: any core that lists the `iob_linux_device_drivers` module in its `sw_modules` has its Linux kernel driver, user-space library, C test suite, and documentation generated from its CSR description at setup time, as detailed in Section 4.3.
- Optional accelerator demo.** A `dma_demo` parameter wires a pair of example AXI-Stream input and output peripherals to the peripheral-Direct Memory Access (DMA) path, adding the clock, clock-enable, and reset signals the pairing needs. It gives a

self-contained example of a DMA-driven, stream-based accelerator running under Linux, and is disabled by default because it exists to illustrate that integration path rather than being required to boot a Linux-capable system.

Because `iob_system_linux` is an ordinary Py2HWSW core, it reuses the framework's two-stage setup/build flow, Verilog generators, deterministic Nix toolchain, and FuseSoC/IP-XACT export without modification; the Linux-specific helper scripts sit in the core's `scripts/` directory and are exported to every derived build directory.

4.2. The `soc_linux.py` Template

`soc_linux.py` is the user-facing entry point of the IOB-SoC-Linux repository. It is a thin Py2HWSW `setup()` function that returns a Core Dictionary whose parent field names `iob_system_linux`, so the template inherits every attribute and file of the framework core described above and only records the deltas specific to a given deployment. In its default form it selects the CPU (`iob_vexriscv`), disables the optional tester subsystem to shorten setup time, and declares five configurable FPGA build targets (AES-KU040-DB-G, Cyclone V GT DK, Zybo Z7, SmartZynq SL, and ZCU104). These are build configurations provided by the template; the experimental validation reported in Section 6 was performed on the AES-KU040-DB-G board.

The template exposes two distinct override channels:

- **Pre-processed overrides**, passed through the parent's `system_attributes` field, are interpreted by the `iob_system_linux` scripts before elaboration—for example, peripheral entries whose CSR maps must feed the DTS and driver generators.
- **Raw overrides**, given as top-level ports, wires, subblocks, and snippets keys, are appended to or replace the corresponding inherited attributes verbatim, without further processing.

Adding a custom IP core to the system therefore reduces to a single subblocks entry marked as a peripheral: the register interface, address space, and interrupt line declared there are the primary description from which the pipeline of Section 4.3 derives the RTL, Linux driver, DTS snippet, C test suite, documentation, and IP-XACT/FuseSoC package.

The repository ships an optional `cache_demo` configuration that illustrates this override mechanism on a non-trivial extension. When enabled, it appends an `iob_cache` and an `iob_dma` peripheral, the AXI wiring that connects them, a widened `iob_axi_full_xbar` carrying one extra subordinate port, and the Verilog snippets that tie off the cache invalidation and write-through-buffer control signals—all expressed as overrides in `soc_linux.py` rather than as edits to the framework core. As with `dma_demo`, it is disabled by default because its purpose is to demonstrate how a peripheral accelerator is added through `soc_linux.py`, not because it is needed to boot a Linux-capable system.

4.3. Driver and Metadata Generation Pipeline

The primary contribution of IOB-SoC-Linux is the expansion of Py2HWSW with the `iob_linux_device_drivers` module, a toolset that automates the generation of Linux kernel drivers, DTS files, test suites, and documentation directly from hardware CSR descriptions, removing the repetitive code otherwise written by hand to bring a new IP core up under Linux. Figure 2 gives an overview of this generation flow: a single Py2HWSW description of a peripheral—its CSR map, address space, and interrupt lines—is the primary description of the register interface, supplemented by a little integration metadata. From it, two distinct generators run in one pass: the Py2HWSW hardware generator produces (i) the Verilog RTL and CSR decoding logic, while the `iob_linux_device_drivers` module (with its helper scripts) produces the Linux software artifacts—(ii) per-peripheral DTS snippets (`create_device_tree_files.py`) that `iob_system_dts.py` merges into the

system-wide Device Tree; (iii) the Linux kernel module and user-space library exposing `sysfs`, `/dev`, and `ioctl` interfaces (`create_peripheral_device_drivers.py`); (iv) C test suites (`create_peripheral_tests.py`); (v) $\text{\LaTeX}$ driver documentation (`create_driver_documentation.py`); and (vi) an IEEE 1685 IP-XACT description (`ipxact_gen.py`) together with a VLNv-tagged FuseSoC .core package (`fusesoc.py`). A dashed buildroot-update loop recompiles the driver and rebuilds the Buildroot rootfs for on-target validation. The current generator targets *register-access* peripherals—those whose software interface can be expressed through register addresses, widths, and access modes. Within that scope it emits a complete, loadable driver and its supporting artifacts; peripheral-specific behaviour is added by hand, as detailed in Section 4.5.

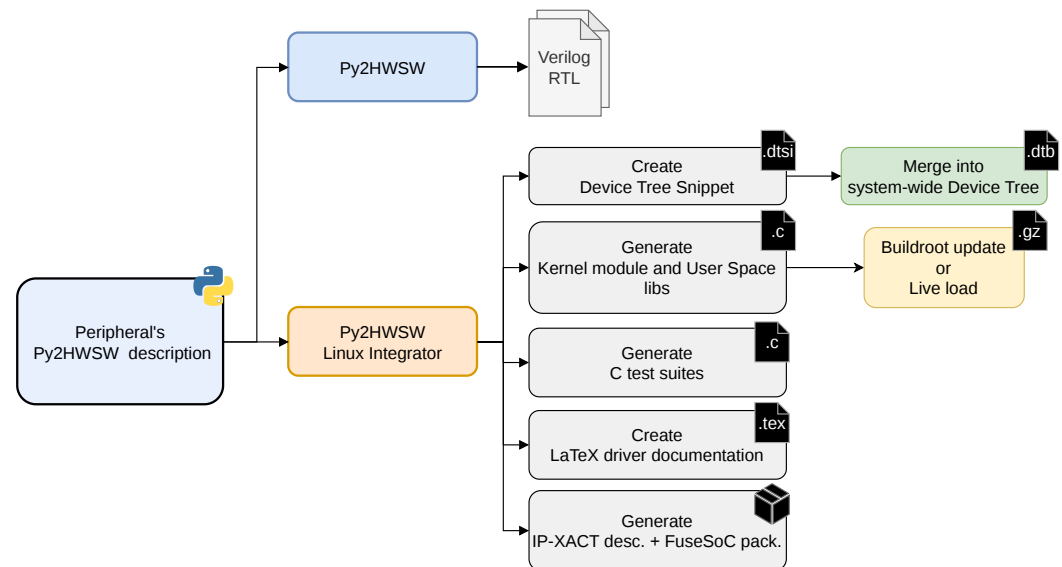


Figure 2. Automated Linux-support generation pipeline, from a Py2HWSW peripheral description to the generated RTL and Linux software artifacts.

Driver generation is opt-in per peripheral. A core requests it by adding an `iob_linux_device_drivers` entry to its `sw_modules` list, optionally supplying a Device Tree compatible string, extra DTS properties such as the peripheral's PLIC interrupt line, and a flag requesting interrupt support. At setup time Py2HWSW calls the module with a handle to the issuing core; the module produces software only—it generates no hardware and instantiates nothing—and writes its output under `software/linux/` in the core's build directory. Because the request is part of the peripheral description, adding a driver-equipped IP core through `soc_linux.py` needs no change to the framework core of Section 4.1. The bundled `iob_timer` peripheral makes this request, and, being one of the system's real, always-present peripherals rather than a standalone demonstration, is used as the running example for the rest of this section (Listing 1).

The single input is the peripheral's `iob_csrs` block—the declarative register list already present in every Py2HWSW core to build its register file, giving one entry per register as a few named fields (name, access mode, width, reset value, description) rather than as procedural code. The generator flattens any register groups into a flat list, appends the implicit 24-bit read-only version register that every Py2HWSW core carries, and reads three attributes per register: its name, its access mode (readable, writable, or both), and its width, which is resolved through the core's Verilog parameters to 8, 16, or 32 bits and mapped to the matching C integer type. Registers that describe an address range rather than a single location are kept out of the `sysfs` view.

From this list the generator specializes a fixed driver skeleton by string substitution; there is no separate template engine. The kernel module, `<name>_main.c`, is a Linux

platform driver: its `of_device_id` table carries the compatible string, and its probe routine maps the register window with `devm_ioremap_resource`, allocates a character device and class, creates the sysfs attribute files, and—when interrupt support was requested—claims the line returned by `platform_get_irq` and installs a handler that wakes a wait queue. The read, write, and `ioctl` file operations are built as switch statements with one arm per register, each keyed on the generated `<CORE>_CSRS_<REG>_ADDR` macro and using the matching `_W` width macro with the `iob_data_read_reg / iob_data_write_reg` accessors. These address and width macros are the same ones Py2HWSW already generates for bare-metal access, so the driver and the firmware share a single definition of the register map.

Listing 1 shows the transformation for the bundled `iob_timer`: three of its seven register entries, the Device Tree node generated from them, and two arms of the generated read handler; the address-map placeholders shown are resolved later against the assembled system, and the elided arms follow the same pattern as the one shown for `data_low`. That seven-register description expands to approximately 1,600 lines of integration artifacts—kernel module, user-space library, C test suite, Device Tree snippet, and L^AT_EX documentation—illustrating the volume of repetitive, mechanically derivable code removed from the developer workflow rather than serving as a figure of merit in itself. None of it is written by hand, and the developer adds only the peripheral-specific behaviour noted in Section 4.5. Section 6.3 breaks this figure down by artifact and sets it against the effort of an equivalent manual integration.

Listing 1. Generation for the bundled `iob_timer` peripheral: input register description (a), generated Device Tree node (b), and generated read file operation (c).

```

1  /* (a) Input: 3 of the 7 entries in the iob_csrs block of iob_timer.py.
2     A read-only 24-bit "version" register is appended implicitly. */
3  {"name": "enable", "mode": "W", "n_bits": 1, "log2n_items": 0},
4  {"name": "data_low", "mode": "R", "n_bits": 32, "log2n_items": 0},
5  {"name": "interrupt_data_low", "mode": "W", "n_bits": 32, "log2n_items": 0},
6
7  /* (b) Generated peripheral node, timer0.dtsi */
8  timer0: iob_timer@/*timer0_BASE_MACRO*/ {
9      compatible = "iobundle,iob_timer";
10     reg = <0x/*timer0_BASE_MACRO*/ 0x/*IOB_TIMER_CSRS_ADDR_RANGE_MACRO*/>;
11     interrupt-parent = < &PLIC0 >; interrupts = <1>;
12 };
13
14 /* (c) Generated read() file operation, iob_timer_main.c */
15 switch (*ppos) {
16 case IOB_TIMER_CSRS_DATA_LOW_ADDR:
17     value = iob_data_read_reg(iob_timer_data.regbase,
18                             IOB_TIMER_CSRS_DATA_LOW_ADDR, IOB_TIMER_CSRS_DATA_LOW_W);
19     size = (IOB_TIMER_CSRS_DATA_LOW_W >> 3); /* bits -> bytes */
20     break;
21 /* ... one arm per readable register (data_high, version) ... */
22 case IOB_TIMER_INTERRUPT_ADDR: /* emitted only with interrupt support */
23     iob_timer_irq_received = 0;
24     if (wait_event_interruptible(iob_timer_wait_queue,
25                               iob_timer_irq_received != 0))
26         return -ERESTARTSYS;
27     value = 1; size = 4; break;
28 default:
29     return -EACCES;
30 }

```

Regenerating the whole system after a register is added or changed takes about 35 s and edits no generated file by hand: a single new entry in an `iob_csrs` list propagates through one `make setup` to the kernel driver, all three user-space interfaces, and the test suite, including its negative checks.

All three user-space interfaces are generated from that same register list. They are emitted together because each suits a different consumer: the `sysfs` interface exposes one file per scalar register under `/sys/class/`, with permissions taken from the register's access mode, and is convenient for shell scripts and simple configuration or status monitoring; the `/dev` interface maps register offsets onto `read` and `write` at a file position set by `lseek`, suiting register-oriented C applications and bulk access; and the `ioctl` interface assigns one command code per register under a per-peripheral magic number, giving a typed, structured entry point for applications that prefer it. A generated user-space library gives all three the same `<name>_csrs_set_<reg>() / get_<reg>()` API, so an application chooses its interface at compile time (`make IF=sysfs|dev|ioctl`) without source changes; the marginal cost of generating all three from the register list is small. The pipeline also emits a cross-compilation `Makefile` (RV32IMAC, ilp32), the driver `.mk` fragment for the in-kernel build, and a `README`.

Three companion scripts complete the artifact set. `create_device_tree_files.py` writes the per-peripheral DTS snippet—a `&soc` node carrying the compatible string, the `reg` range as address-map placeholders, and any extra properties such as the `interrupt-parent/interrupts` pair—which `iob_system_dts.py` then merges into the system-wide Device Tree. `create_peripheral_tests.py` emits a C test suite that exercises every register through the common API and adds negative checks, such as writing a read-only register or reading a write-only one. `create_driver_documentation.py` produces the $\text{\LaTeX}$ section documenting the module, its interfaces, and its tests, keeping the software description up to date as the hardware changes.

The scope and limitations of this generator, and the robustness properties of the code it emits, are discussed in Section 4.5.

To support rapid iteration, the `buildroot-update` workflow automates driver re-compilation and integration into the Buildroot rootfs, allowing hardware changes to be validated on physical hardware without manual intervention. Driver loading and unloading is scripted via `rmmmod/insmmod` sequences, enabling repeatable test execution on the target system.

In parallel with the software artifacts, the `fusesoc.py` and `ipxact_gen.py` exporters produce standardized packaging metadata for a packaged core, as summarized in Figure 3. `fusesoc.py` emits a FuseSoC `.core` manifest carrying an `iobundle:py2hws: name: version Vendor:Library:Name:Version (VLNV)` identifier, file-sets, and tool-specific targets, while the companion `ipxact_gen.py` exporter emits an IEEE 1685 IP-XACT description with the core's parameters, ports, bus interfaces, and register map. FuseSoC resolves inter-core dependencies from the VLNV graph and drives the downstream simulation, lint, and synthesis tools, while the IP-XACT metadata is intended for import into third-party environments such as Vivado IP Integrator and Intel Platform Designer. This work packages and checks IOB-Cache, IOB-UART16550, and IOB-Eth in this way.

4.4. Build-Flow Integration

At the build level, the template adds a `linux_targets.mk` fragment and Linux-specific Make targets on top of the inherited Py2HWSW build system. The `kernel-module-rebuild` target cross-compiles the generated user-space binaries and the kernel module against the `iob-linux` tree, synchronizes the build directory to the target board, and hot-reloads the driver over the serial console with an `rmmmod/insmmod` sequence; companion targets rerun the generated test suites and, via the `buildroot-update` loop of Section 4.3, rebuild the Buildroot root filesystem. The Nix shell inherited from Py2HWSW is extended with the

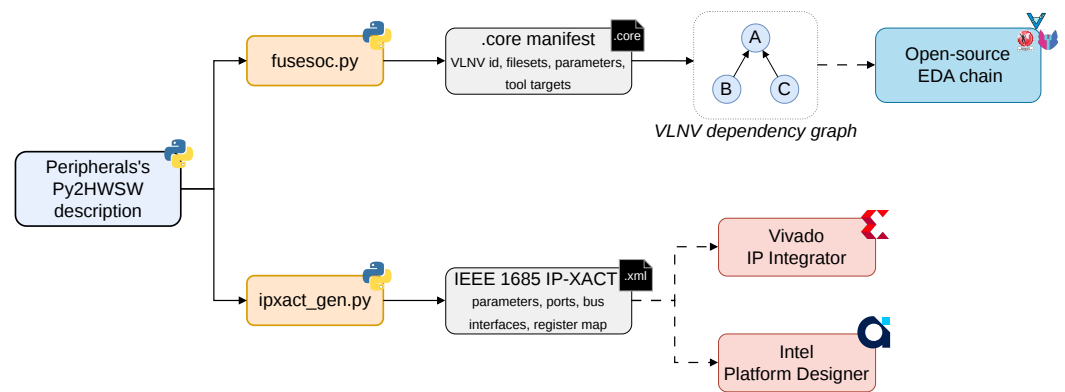


Figure 3. IP-XACT and FuseSoC packaging flow for a packaged peripheral core.

RISC-V Linux cross-toolchain, OpenSBI, and Buildroot so that the entire hardware-to-rootfs flow remains reproducible from a single environment.

4.5. Scope and Limitations of the Generator

The generator's semantic knowledge of each peripheral is limited to register addresses, widths, and access modes. Within that scope it produces a complete, loadable *register-access* driver together with its user-space library, tests, and documentation, but it does not model register side effects, first-in-first-out ports, self-clearing bits, or multi-register control sequences; behaviour of that kind is added by hand on top of the generated module. Interrupt support is likewise only a starting point: the request adds the handler, the wait queue, and a blocking wait operation, but the handler body currently follows the `iob_timer` convention of clearing a threshold register and must be adapted for peripherals that acknowledge interrupts differently. The current contribution therefore automates the *structural* integration of interrupt-capable register-mapped peripherals—handler registration, PLIC line wiring, the wait queue, and the Device Tree `interrupts` properties—rather than the *semantic* specification of each peripheral's interrupt-acknowledgement behaviour, which remains the boundary of the automation. Widening this coverage—richer register semantics in the Py2HWSW description and fully general interrupt handling—is the main line of ongoing work on the pipeline.

The robustness of the emitted code is bounded in a similar way. The generated read and write handlers validate the incoming register offset against the generated address macros and reject out-of-range or wrong-direction accesses with `-EACCES`, and the generated C test suite exercises these negative paths explicitly (for example, writing a read-only register or reading a write-only one). Concurrent access from multiple processes, 64-bit host builds, and hardening against interrupt races are not addressed by the generated code and, where required, belong to the hand-written peripheral-specific layer.

5. IOB-SoC-Linux

5.1. SoC Architecture

As illustrated in Figure 4, IOB-SoC-Linux is built around a RISC-V processor subsystem that currently supports both VexRiscv and VexiiRiscv cores, spanning a range of performance and resource points. Both implement the RV32IMAC ISA with an Sv32 MMU, satisfying the platform requirements of Section 3.1. Both cores offer their own internal CLINT and PLIC implementations, but IOB-SoC-Linux does not use them: keeping them internal would tie the `mtime/mtimecmp`, `msip`, and `interrupt-source` signals inside each core's own wrapper, so swapping in a different RISC-V core would require modifying that wrapper to route the same signals out to the rest of the system. Instead, `iob_system`

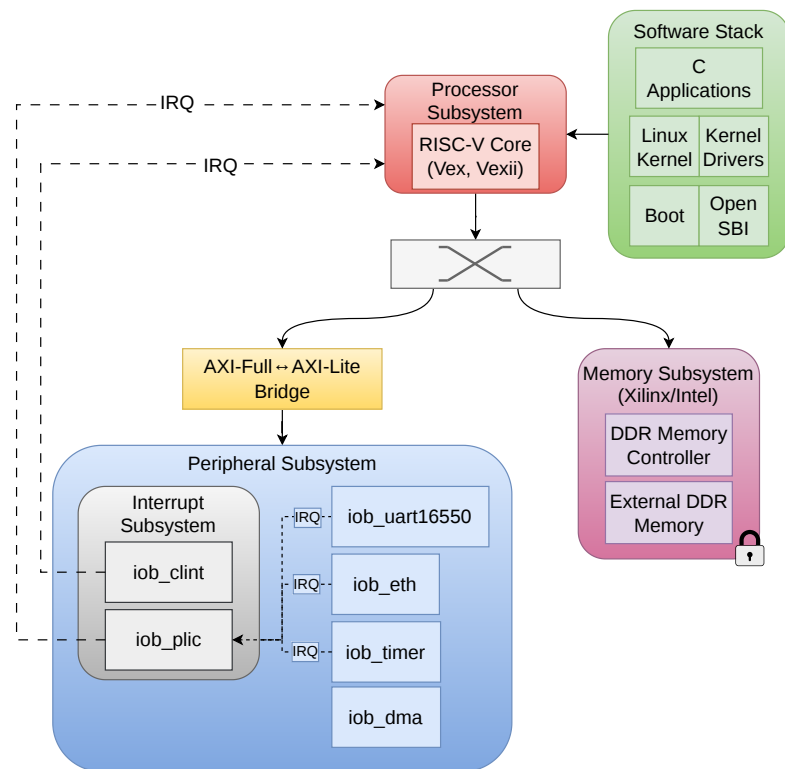


Figure 4. Topview of the IOb-SoC-Linux system default template

instantiates the CLINT and PLIC as external `iob_clint` and `iob_plic` peripherals sitting outside the CPU wrapper, so every core connects to them through the same fixed set of external interrupt lines, with no core-specific wrapper changes needed to support a new one; this also matches the upstream OpenSBI and Linux kernel bindings:

- `iob_clint` implements the machine timer (`mtime/mtimecmp`) and software-interrupt (`msip`) facilities behind the SBI timer and inter-processor-interrupt services, and is probed unmodified through the upstream `riscv,clint0` Device Tree binding.
- `iob_plic` aggregates and prioritizes up to 31 device interrupt lines and presents independent machine- and supervisor-mode contexts to the core, using the upstream `riscv,plic0` binding.

The peripheral subsystem provides the following IP cores; those with a mainline Linux equivalent are additionally checked against it:

- **IOb-UART16550** — three instances of an NS16550-compatible UART: a console port bound to the board RS-232 transceiver and an internal loopback pair used for self-test. All three are register-compatible with the Linux 8250/16550 driver and are wired to PLIC sources 3–5. The core is a maintained, mainline-driver-compatible rewrite of the OpenCores `uart16550` [2], contributed back to the community under the MIT licence.
- **IOb-Eth** — a 10/100 Ethernet MAC with a descriptor-driven engine and a dedicated Advanced eXtensible Interface (AXI)-Full master port for moving frames directly to and from main memory. It is validated against the mainline `ethoc` driver and is additionally used by the bootloader and firmware to accelerate image transfers. Like IOb-UART16550, it is an openly licensed, actively maintained successor to a long-standing OpenCores design (`ethmac` [3]).
- `iob_timer` — a free-running system timer on PLIC source 1, available both to bare-metal firmware and as an auxiliary Linux time source.

- `iob_dma` — a general-purpose DMA engine with AXI-Stream endpoints that moves data between streaming peripherals and DRAM without processor involvement, driven from user space through a generated `ioctl` interface.
- **IOb-Cache** [4] — a configurable cache exposing a dedicated control and statistics CSR block (hit/miss counters, line invalidation, and write-through-buffer status); in the optional `cache_demo` configuration it is attached as a peripheral accelerator that the running OS configures and monitors through a generated `sysfs` interface.

The memory subsystem comprises an external DRAM controller exposing 64 MiB of main memory over AXI-Full, a 64 KiB on-chip boot ROM (`iob_bootrom`) holding the two-stage bootloader, an on-chip Static Random-Access Memory (SRAM) that the second stage executes from, an Serial Peripheral Interface (SPI) controller for booting from off-chip flash, and the AXI-Full crossbar that binds them to the processor. Every control-plane peripheral exposes an AXI-Lite slave interface and attaches through an AXI-Full-to-AXI-Lite bridge, which adapts the high-bandwidth interconnect used by the core and memory to the simpler, area-efficient protocol suited to register access, keeping peripheral integration lightweight without sacrificing compatibility with the system interconnect.

Bringing the system up follows a two-stage boot sequence before control passes to Linux. A small pre-bootloader (`iob_system_preboot.S`) executes in place from the boot ROM, copies the main bootloader into the on-chip SRAM, and jumps to it. The main bootloader (`iob_system_linux_boot.c`) then runs from SRAM and loads the firmware images—the Linux kernel, the Device Tree blob, and the `initramfs`—into DRAM over one of three paths: (i) from the host console over IOB-UART16550, via a `uart16550_recvfile()` transfer; (ii) optionally over IOB-Eth, when the build defines `IOB_SYSTEM_LINUX_USE_ETHERNET`, pairing `uart_recvfile_ethernet()` with `eth_rcv_file()` to speed up the transfer; or (iii) directly from the board's SPI flash via `read_flash()`. The flash image is written once by a `program_flash()` call following an initial UART or Ethernet load, so that later boots proceed with no host connection. Once the images are resident in DRAM, the bootloader hands off to OpenSBI in machine mode, which brings up the RISC-V SBI runtime—servicing timer and inter-processor interrupts through the external `iob_clint`—and hands off to the Linux kernel in supervisor mode. The kernel enables the MMU, parses the auto-generated Device Tree to probe each peripheral and bind its generated driver—produced by the `iob_linux_device_drivers` pipeline of Section 4.3—routes device interrupts through the external `iob_plic`, and mounts the Buildroot/BusyBox root filesystem, which finally launches the user-space shell and the generated peripheral test suites. Figure 5 shows the resulting multi-stage boot and execution flow.

6. Experimental Results

This section reports the validation performed and the quantitative results that accompany it.

6.1. FPGA Validation and Resources

IOb-SoC-Linux was validated by choosing a device (`iob_timer`), verifying that the generated RTL builds and closes timing on real hardware, boots a full Linux distribution, and its generated driver's (`sysfs`, `/dev`, and `ioctl`) interfaces all pass their test suites, including negative permission checks correctly rejected with `-EACCES`, meaning a correct operation under that running OS.

On this basis, IOB-SoC-Linux was validated across two RISC-V cores, VexRiscv and VexiiRiscv, on three of the current FPGA boards supported by Py2HWSW, spanning 2 vendors, showing that IOB-SoC-Linux is not tied to a single CPU or a single FPGA vendor. The Xilinx Kintex UltraScale KU040 and Zynq UltraScale+ ZCU104 (XCZU7EV) were built

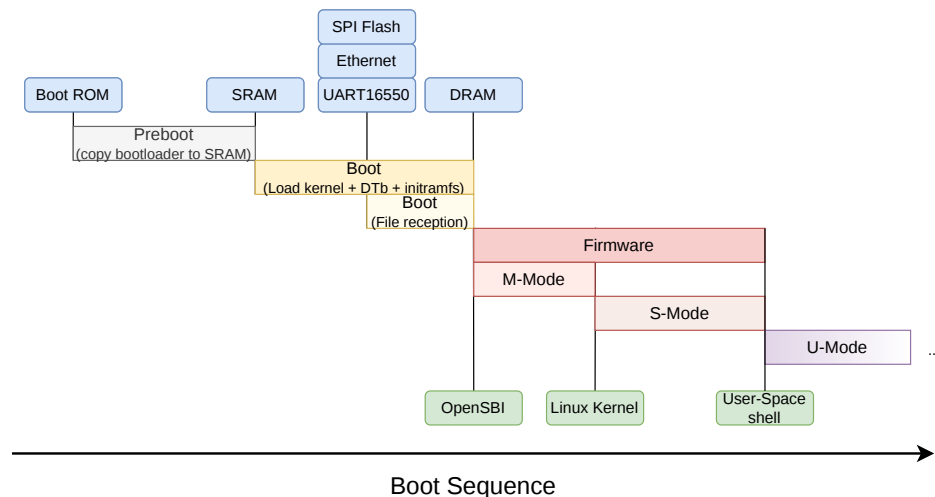


Figure 5. Multi-stage boot and software execution flow of IOB-SoC-Linux, from Boot ROM to the Linux user-space shell.

with Vivado 2020.2, while the Altera Cyclone V GT DK (5CGTFD9E5F35C7) was built with Quartus Prime 20.1.

Table 1 reports post-place-and-route FPGA utilization and maximum clock frequency for all six board/core combinations, each at its own maximum possible clock frequency. The Xilinx boards, KU040 and ZCU104, report LUTs, while the Altera board, Cyclone V, reports ALMs. Maximum frequency was found by bisecting the clock constraint to the tightest period that still meets timing. Both cores reach the same frequency ceiling on a given board, set by the shared bus fabric and DDR clock-domain-crossing logic rather than by either core. The KU040 figures reflect each core's default configuration (branch prediction enabled, Section 6.2), which closes timing at 100 MHz rather than the 125 MHz achievable without it. ZCU104's lower utilization and higher maximum frequency are both consistent with its external DRAM being served by the board's hardened Processing System rather than by a soft memory controller competing with the design for fabric resources. The smaller KU040-versus-Cyclone V gap follows the same pattern at a smaller scale, reflecting each vendor's own logic primitive and soft DDR controller implementation on a different process node. The WNS and TNS values, reported for each board/core combination, were measured on the same run that established that board's Fmax; a TNS of 0 ns confirms each result is a genuine pass, not a rounding artifact.

Table 1. FPGA utilization and maximum clock frequency on all three boards, for both supported cores.

Metric	KU040		Cyclone V		ZCU104	
	VexRiscv	VexiiRiscv	VexRiscv	VexiiRiscv	VexRiscv	VexiiRiscv
Logic cells (LUTs or ALMs)	23,046 (9%)	24,482 (10%)	16,940 (15%)	19,802 (17%)	10,594 (5%)	12,042 (5%)
Registers (FFs)	25,249 (5%)	25,745 (5%)	27,632	32,659	8,072 (2%)	8,557 (2%)
BRAM (Kb)	1,674 (8%)	1,710 (8%)	820 (7%)	858 (7%)	738 (7%)	774 (7%)
DSP	7 (0.4%)	7 (0.4%)	4 (1%)	4 (1%)	4 (0.2%)	4 (0.2%)
Fmax (MHz)	100	100	80	80	200	200
WNS (ns)	+1.09	+1.30	+0.01	+0.07	+0.13	+0.27

The same RTL synthesizes cleanly through the open-source Yosys ASIC flow to a gate-level netlist, showing that IOB-SoC-Linux is not limited to FPGA toolchains.

6.2. Performance

To quantify IOB-SoC-Linux's performance on standard benchmarks, we measured it on the KU040 board, running Linux 5.15 and a Buildroot user space at the board's maximum system clock, for both supported RISC-V cores. Table 2 reports the resulting CoreMark, Dhrystone, and Ethernet throughput, all measured from bundled Buildroot packages running directly on the target.

VexRiscv runs with DYNAMIC_TARGET branch prediction and 4 KB, direct-mapped I/D caches; VexiiRiscv runs with BTB/GShare/RAS prediction and 8 KB, 2-way I/D caches with LSU bypass; both at the KU040's 100 MHz closing frequency. VexiiRiscv's Ethernet driver remains unavailable in this configuration, a pre-existing cache-coherency limitation unrelated to clock frequency.

Table 2. Runtime measurements on the KU040 board, for both supported cores.

Metric	VexRiscv	VexiiRiscv
CoreMark / MHz	2.02	1.12
Dhrystone (DMIPS / MHz)	0.99	0.75
Ethernet TCP, Tx / Rx (Mbit/s)	10.1 / 5.97	—
Ethernet UDP, Tx (Mbit/s)	2.85 (0% loss)	—

6.3. Generated vs. Manual Peripheral Integration

Table 3 shows, for three bundled peripherals of increasing register count, how far a short hardware register description expands into the software artifacts of Section 4.3. Each peripheral is described by an `iob_csrs` block of 7–33 register entries (58–298 lines). From that description alone, the pipeline emits 1,162–3,394 lines of C (the kernel module, the three user-space interface libraries, and the C test suite) plus the Device Tree node, measured with `cloc` (a source-code line-counting tool). None of it is written by hand. `iob_eth`, the largest entry in the table, is the same peripheral whose generated driver carried the `iperf3` traffic reported in Section 6.1.

Without this pipeline, every one of these lines is work a developer must do by hand, and keep consistent with the RTL as it evolves: write the kernel driver, its user-space libraries and tests, and the matching Device Tree node. The bring-up tutorial's own starting template, a hand-written driver for `iob_spi` (`iob_spi_main.c`), shows the size of that task. It is 334 lines of C for the `sysfs` interface alone, plus a thirteen-step manual procedure spanning the driver, the Device Tree, and the kernel and Buildroot build integration.

Table 3. Generation output for three bundled peripherals of increasing register count.

Peripheral	Registers	Python	Generated C	DT
<code>iob_timer</code>	7	58	1,162	18
<code>iob_uart16550</code>	8	74	1,265	22
<code>iob_eth</code>	33	298	3,394	47

6.4. Positioning Relative to Existing Frameworks

Table 4 summarizes how IOB-SoC-Linux relates to the frameworks discussed in Section 2, on integration-automation and metadata-generation capabilities rather than overall SoC capability, where these systems have different and largely non-overlapping goals. Each capability column records whether the framework generates that artifact automatically for a newly attached register-access peripheral: a system-wide Device Tree, a Linux kernel driver with its C test suite and documentation, and standardized IP-XACT/FuseSoC packaging. Among the systems surveyed, none automates all three: Device Tree generation appears in isolation (Chipyard, and LiteX for its own peripheral catalog), and OpenTitan's `reggen`

comes closest on the software side but targets bare-metal firmware rather than the Linux kernel. IOB-SoC-Linux is, on this axis, the only surveyed system that produces all three automatically, while also supporting multiple FPGA boards, an ASIC flow, and more than one Linux-capable RISC-V core.

Table 4. Positioning of IOB-SoC-Linux relative to representative open-source RISC-V SoC frameworks and Linux-capable platforms discussed in Section 2, on integration-automation and metadata-generation capabilities only.

Framework	SW target	Input description	System Device Tree	Linux driver, tests & docs	IP-XACT / FuseSoC	RISC-V core(s)
LiteX [5]	Linux, bare-metal	Migen (Python DSL)	Partial	No	No	Configurable soft cores
Chippyard [6]	Linux, bare-metal	Chisel (Scala)	Yes	No	No	Configurable (Rocket, BOOM)
X-HEEP [7]	Bare-metal, RTOS	SystemVerilog	No	No	No	Single core
Cheshire [8]	Linux	SystemVerilog	No	No	No	CVA6 (fixed)
Basilisk [9]	Linux	SystemVerilog (hand-assembled)	No	No	No	CVA6 (fixed)
HULK-V [11]	Linux	SystemVerilog	No	No	No	Host + PMCA (fixed)
OpenTitan [12]	Firmware	SystemVerilog + HJSON (reggen)	N/A	No	No	Ibex (fixed)
Arm Socrates [13]	— (IP configurator)	Vendor IP catalog	No	No	IP-XACT export (commercial)	—
IOB-SoC-Linux (this work)	Linux, bare-metal	Python / JSON (Py2HWSW)	Yes	Yes	Yes	Configurable (VexRiscv, VexiiRiscv)

7. Conclusion

This paper presented IOB-SoC-Linux, an open-source framework for generating Linux-capable RISC-V SoCs with automated hardware/software co-design support. Built on Py2HWSW, the framework provides a reproducible environment for integrating and validating custom IP cores under a full Linux software stack, in contrast with bare-metal-focused alternatives that leave system assembly and software integration to the user.

The primary contribution, the IP core Linux integration tool, generates a register-access Linux kernel driver together with its Device Tree snippet, user-space library, C test suite, and documentation directly from a peripheral's hardware CSR description, so that bringing a new register-access IP core into a Linux environment no longer requires the register map, Device Tree, and driver to be written and kept consistent by hand; peripheral-specific semantics and interrupt-acknowledgement behaviour are added on top of the generated module. Peripheral IP cores were exercised through their generated drivers, and those with a mainline Linux equivalent—IOB-UART16550 and IOB-Eth—were additionally checked against the upstream 8250 and `ethoc` drivers, confirming both functional correctness and ecosystem compatibility. The verification strategy spans RTL linting, simulation coverage analysis, system-level simulation, and on-board FPGA validation on a Xilinx Kintex UltraScale board, giving confidence across the full development stack.

Beyond the framework itself, the three packaged peripheral cores—IOB-Cache, IOB-UART16550, and IOB-Eth—are contributed back to the open-source hardware community as MIT-licensed, IP-XACT/FuseSoC-described blocks that can be reused outside IOB-SoC-Linux, with IOB-UART16550 and IOB-Eth providing maintained, mainline-driver-compatible successors to the classic OpenCores `uart16550` and `ethmac` designs.

Future work will extend processor support to further RISC-V cores—NaxRiscv and Ibex integration is already under way—complete the automation of interrupt handling and add richer register semantics to the driver generation pipeline, and extend IP-XACT/FuseSoC packaging beyond the current three peripheral cores to improve interoperability with third-party EDA toolchains.

Author Contributions: Conceptualization, A.N. and J.T.d.S.; methodology, A.N., J.B.R. and J.A.; software, A.N., J.B.R. and J.A.; validation, A.N., J.B.R. and J.A.; formal analysis, A.N. and J.T.d.S.; investigation, A.N., J.B.R. and J.A.; resources, J.T.d.S.; writing—original draft preparation, A.N. and J.B.R.; writing—review and editing, J.B.R., J.A. and J.T.d.S.; supervision, J.T.d.S.; project administration, J.T.d.S.; funding acquisition, J.T.d.S. All authors have read and agreed to the published version of the manuscript.

Funding: This project was mainly funded by the NLnet NGI0 Core Fund, with support from the European Commission’s Next Generation Internet programme, grant agreement No. 101092990. J.B.R. and J.A. acknowledge Fundação para a Ciência e a Tecnologia (FCT) support through grant BI|2025/772–Proj. 2023.16747.ICDT (SMARTChip, DOI: <https://doi.org/10.54499/2023.16747.ICDT>, Ref^a LISBOA2030-FEDER-00692100), and projects UID/6486/2025, UID/PRR/6486/2025, UID/PRR2/06486/2025, UID/50021/2025, UID/PRR/50021/2025.

Data Availability Statement: The IOB-SoC-Linux framework described in this paper—the SoC template, the `iob_linux_device_drivers` tooling, and the associated build flow—is open-source and publicly available at <https://github.com/IObundle/soc-linux>. The underlying Py2HWSW hardware/software co-design framework is available as referenced in [18]. No new datasets were generated beyond the source code and generated artifacts hosted in these repositories.

Conflicts of Interest: Authors Artur Nóbrega and Jaime Aguiar have a connection to the company IObundle, Lda. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

1. Antunes, P.N.D.M. Linux capable RISC-V CPU for IOB-SoC. Master’s thesis, 2022.
2. OpenCores. UART 16550 Core. Available at: <https://opencores.org/projects/uart16550>. Accessed: 2026-09-09.
3. OpenCores. Ethernet MAC 10/100 Mbps Core (ethmac). Available at: <https://opencores.org/projects/ethmac>. Accessed: 2026-09-09.
4. Roque, J.V.; Lopes, J.D.; Véstias, M.P.; de Sousa, J.T. Iob-cache: A high-performance configurable open-source cache. *Algorithms* **2021**, *14*, 218.
5. Kermarrec, F.; Bourdeauducq, S.; Le Lann, J.C.; Badier, H. LiteX: an open-source SoC builder and library based on Migen Python DSL, 2020, [arXiv:cs.AR/2005.02506].
6. Amid, A.; Biancolin, D.; Gonzalez, A.; Grubb, D.; Karandikar, S.; Liew, H.; Magyar, A.; Mao, H.; Ou, A.; Pemberton, N.; et al. Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs. *IEEE Micro* **2020**, *40*, 10–21. <https://doi.org/10.1109/MM.2020.2996616>.
7. Schiavone, P.D.; Machetti, S.; Peón-Quirós, M.; Miranda, J.; Denking, B.; Müller, T.C.; Rodríguez, R.; Nasturzio, S.; Atienza, D. X-HEEP: An Open-Source, Configurable and Extendible RISC-V Microcontroller. In Proceedings of the Proceedings of the 20th ACM International Conference on Computing Frontiers (CF ’23), Bologna, Italy, 2023. <https://doi.org/10.1145/3587135.3591431>.
8. Ottaviano, A.; Benz, T.; Scheffler, P.; Benini, L. Cheshire: A Lightweight, Linux-Capable RISC-V Host Platform for Domain-Specific Accelerator Plug-In. *IEEE Transactions on Circuits and Systems II: Express Briefs* **2023**, *70*, 3777–3781. <https://doi.org/10.1109/TCSII.2023.3289186>.
9. Scheffler, P.; Wistoff, N.; Zaruba, F.; Benini, L.; Gürkaynak, F.K. Basilisk: An End-to-End Open-Source Linux-Capable RISC-V SoC in 130nm CMOS, 2024, [arXiv:cs.AR/2406.15107].

10. Sauter, P.; Scheffler, P.; Wistoff, N.; Benz, T.; Benini, L.; Gürkaynak, F.K. Basilisk: A 34 mm² End-to-End Open-Source 64-bit Linux-Capable RISC-V SoC in 130nm BiCMOS, 2025, [arXiv:cs.AR/2505.10060].
11. Valente, L.; Tortorella, Y.; Sinigaglia, M.; Tagliavini, G.; Capotondi, A.; Benini, L.; Rossi, D. HULK-V: a Heterogeneous Ultra-low-power Linux capable RISC-V SoC, 2022, [arXiv:cs.AR/2211.14944].
12. lowRISC. OpenTitan: Open Source Silicon Root of Trust. Available at: <https://opentitan.org/>, 2026. Accessed: 2026-08-26.
13. Arm Limited. Arm Socrates User Guide. Available at: <https://developer.arm.com/documentation/101399/latest>, 2026. Accessed: 2026-08-26.
14. Kindgren, O. A Scalable Approach to IP Management with FuseSoC. In Proceedings of the Open-Source Design Automation (OSDA) Workshop, Florence, Italy, 2019.
15. RISC-V International. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 20240411. Technical report, RISC-V International, 2024. Accessed: 2026-09-07.
16. RISC-V International. RISC-V Supervisor Binary Interface Specification, Version 2.0. Technical report, RISC-V International, 2023. Accessed: 2026-09-07.
17. RISC-V Software. OpenSBI: RISC-V Open Source Supervisor Binary Interface. Available at: <https://github.com/riscv-software-src/opensbi>, 2026. Accessed: 2026-09-07.
18. Anonymous. Py2HWSW. Available at: [REPOSITORY_LINK_CENSORED_FOR_REVIEW], 2026.